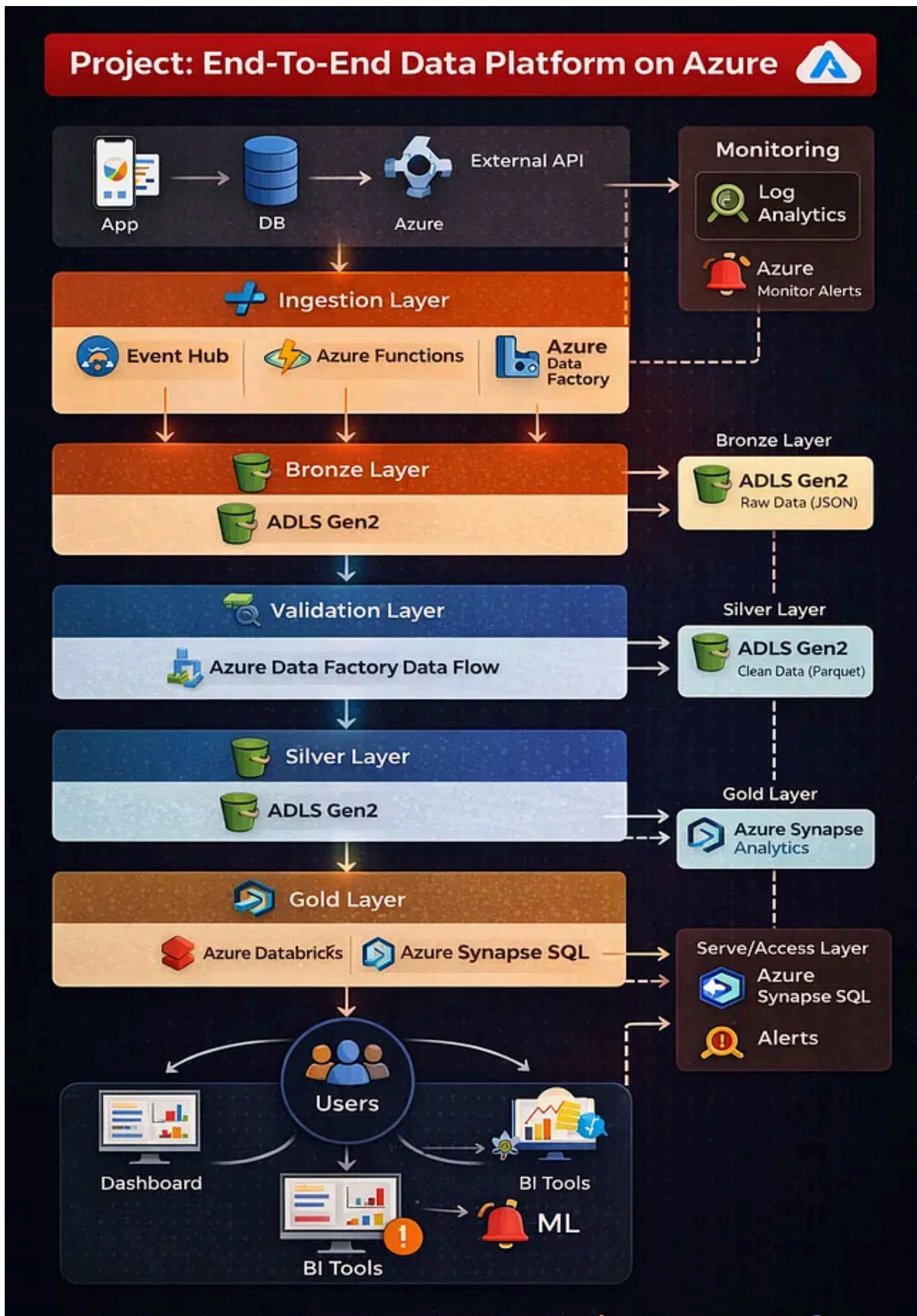




How I Built a Production-Ready End-to-End Data Platform on Azure Using AI-Assisted Development



Anurag Sharma 5 min read · Jun 8, 2026



5



In today's rapidly evolving AI-driven engineering landscape, the role of a Data Engineer is no longer limited to just writing ETL jobs or managing data infrastructure. Instead, it has expanded to encompass understanding business requirements, breaking them into actionable tasks, and accelerating implementation through AI-assisted development.

One of the biggest challenges engineers face is translating high-level Business Analyst (BA) requirements into structured, implementable data pipelines. Often, requirements come in abstract forms — “build a data platform,” “enable analytics,” or “support real-time insights.” The real skill lies in decomposing these into smaller, logical, and sequential tasks that can be developed, tested, and scaled independently.

In this blog, we take a practical, end-to-end approach.

We start with a single business goal:



Build a scalable data platform on Azure



Then, we break it down into 10 interconnected pipelines (tasks) — each representing a real-world step in a modern data engineering workflow:

- From data ingestion
- To validation and transformation
- To aggregation and warehouse loading
- All the way to monitoring and data access

Each pipeline is presented as:

- A clear BA requirement
- A technical implementation flow
- The Azure services used
- And most importantly, an AI prompt to help you generate code faster

Why This Approach Matters

This structured breakdown helps you:

- Think like a Data Engineer + Architect
- Understand how real-world data platforms are built step-by-step

- Learn how to collaborate effectively with business stakeholders
- And leverage AI as a productivity multiplier, not just a coding tool

Business Context:

We are building a centralized analytics platform for a digital product company that collects:

- User activity logs
- Transactions
- External API data

The goal is to enable:

- Real-time monitoring
- Historical analytics
- ML readiness

Overall High-Level Flow

```
Sources (App, API, DB)
↓
Ingestion Layer (Event Hub, Azure Functions, Azure Data Factory)
↓
Raw Storage (ADLS Gen2 - Bronze)
↓
Processing Layer (Azure Databricks / Synapse)
↓
Curated Storage (ADLS - Silver/Gold)
↓
Serving Layer (Synapse Analytics / Serverless SQL)
```

↓

Consumption (Power BI, ML, Alerts)

Pipeline 1: Data Ingestion (Batch + API + DB)

BA Requirement

We need to ingest:

- Application logs (batch upload)
- External API data (daily)
- Transaction DB changes (near real-time)

Azure Stack

- ADLS Gen2 (raw storage)
- Azure Functions (API ingestion)
- Azure Data Factory (CDC from DB)

Flow

API → Azure Functions → ADLS
DB → ADF (CDC) → ADLS
Logs → ADLS upload

Sample Data Instruction

Generate mock API JSON:

```
{ "user_id": 101, "event": "login", "timestamp": "2026-01-01T10:00:00" }
```

Sample AI Prompt

“Generate Azure Function Python code to fetch paginated API data and store it in ADLS Gen2 in JSON format with partitioning by date.”

Pipeline 2: Raw Data Validation

BA Requirement

Ensure all incoming data meets schema and quality standards.

Azure Stack

- Azure Data Factory Data Flow / Databricks
- ADLS
- Azure Monitor Alerts

Flow

```
ADLS (raw) → Data Validation → Pass/Fail → Alerts
```

Sample Data

Include bad records:

```
{ "user_id": null, "event": "", "timestamp": "invalid" }
```



Sample AI Prompt

“Write PySpark code in Azure Databricks to validate schema, null checks, and timestamp formats with error logging.”

Pipeline 3: Data Lake (Bronze → Silver)

BA Requirement

Clean and standardize raw data into a structured format.

Azure Stack

- Azure Databricks
- ADLS (partitioned parquet)

Flow

Raw (JSON) → Databricks → Cleaned (Parquet)

Sample Transformation

- Convert timestamp → ISO
- Remove duplicates



Sample AI Prompt

“Generate PySpark code in Databricks to transform JSON data into partitioned Parquet format by date in ADLS.”

Pipeline 4: Incremental Processing

BA Requirement

Process only new data (avoid full reloads).

Azure Stack

- Azure Data Factory incremental load
- ADLS partitions

Flow

New Data → Incremental Job → Append to Silver

Sample Data

/data/year=2026/month=01/day=01/

Sample AI Prompt

“Write an Azure Data Factory pipeline to process only newly added files in ADLS using watermark or incremental logic.”

Pipeline 5: Data Enrichment

BA Requirement

Enhance data with external datasets (e.g., geo, user metadata).

Azure Stack

- Azure Databricks
- ADLS

Flow

Silver Data + Lookup Data → Join → Enriched Data

Sample Data

```
{ "user_id": 101, "country": "US" }
```



Sample AI Prompt

“Create a PySpark job in Databricks to join transactional data with a lookup dataset on user_id.”

Pipeline 6: Aggregation (Gold Layer)

BA Requirement

Create business metrics:

- Daily active users
- Transactions per hour

Azure Stack

- Databricks / Synapse SQL

Flow

Enriched Data → Aggregation → Gold Tables

Example Output

```
date, active_users  
2026-01-01, 1200
```

Sample AI Prompt

“Write a Synapse SQL query to calculate daily active users from event logs.”

Pipeline 7: Data Warehouse Load

BA Requirement

Load curated data into a warehouse for BI.

Azure Stack

- Azure Synapse Analytics
- COPY INTO from ADLS

Flow

ADLS Gold → Synapse

Sample Tables

- users_metrics

- transaction_summary



Sample AI Prompt

“Generate Synapse SQL COPY INTO command and table schema for loading partitioned Parquet data from ADLS.”

Pipeline 8: Orchestration

BA Requirement

Automate the full pipeline with dependencies.

Azure Stack

- Azure Data Factory Pipelines

Flow

Ingestion → Validation → Processing → Aggregation → Load



Sample AI Prompt

“Create an Azure Data Factory pipeline JSON for orchestrating multiple data flows with retries and dependencies.”

Pipeline 9: Monitoring & Alerting

BA Requirement

Track failures, delays, and anomalies.

Azure Stack

- Azure Monitor
- Log Analytics
- Alerts

Flow

Jobs → Logs → Alerts



Sample AI Prompt

“Configure Azure Monitor alerts for failed data pipelines and Databricks jobs with notification setup.”

Pipeline 10: Data Access Layer

BA Requirement

Provide easy access to data for analysts.

Azure Stack

- Synapse Serverless SQL
- Power BI

Flow

Synapse → Power BI Dashboard



Sample AI Prompt

“Generate Synapse external table DDL and sample queries for querying partitioned ADLS data.”

Final Architecture Summary

```
[Sources]
API | DB | Logs
↓
Ingestion Layer
↓
ADLS (Raw/Bronze)
↓
Validation Layer
↓
Processing (Databricks)
↓
ADLS (Silver → Gold)
↓
Synapse Analytics
↓
Power BI / Alerts / ML
```

💡 Why This is Strong for Portfolio

This setup demonstrates:

- Batch + streaming mindset
- Data lake architecture
- Orchestration
- Data quality
- Warehouse integration
- AI usage for building an E2E pipeline

[Data Engineering](#)[Data Engineer](#)[Data Analytics](#)[Big Data](#)[Ai Data Engineering](#)**Written by Anurag Sharma**[Edit profile](#)

125 followers · 5 following

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!